Bit-Vector Encoding of N-Queen Problem *

Qiu Zongyan
Department of Informatics, School of Math. Sciences
Peking University, Beijing 100871, CHINA

Abstract

8-queen problem and its generalization, n-queen problem are well-known examples in the textbooks on elementary programming, data structures, and algorithms. Different methods are proposed to solve these problems, for example, in [1]. In this paper, we present a purely bit-vector encoding of the n-queen problem. It is very natural, simple to understand, and efficient. It involves only bit-wise operations.

1 Well-Known Solutions

8-queen problem is a well-known problem in computer science education. This problem and its generalization, n-queen problem, are widely used in courses and textbooks on programming, data structures, design and analysis of algorithms, as examples in discussions on searching, recursion, back-tracking, etc.

The problem is to put 8 queens (or n queens, in general) on a chess board with 8×8 ($n \times n$) positions, that none of them can attack other. Finding all possible placements is a variant of the problem.

A number of methods are proposed to solve the problems. Two basic methods are (We'll use 8-queen as the example in following discussion):

Direct checking: Suppose there is a queen in the ith row at the jth place, or position (i, j). The positions controlled by this queen include: all positions in the ith row and in the jth column, and all the positions (k, l) where $k - l = i - j$ or $k + l = i + j$. The last two groups are the positions on the two diagonals controlled by the queen at (i, j).

If there are some safely placed queens on the board, and the number of queens is less than 8, for progress, we can checking each empty position, to see if it is safe under above conditions with respect to each of the queens on board. If no safe place for a new queen, we need to take out a queen from the board and try other possibilities.

This procedure can be arranged into a series of tests, by first trying to set a queen in the first row, and then a queen in the second row, and so on. If there is no safe position for a queen in the ith row, back-tracking is needed.

Controlled-array method: The drawback of *direct-checking* is the repeated calculation of the checking conditions (with respect to each queen on the board). These re-calculations ought to be avoided.

A improved method uses three arrays *column*, *left* and *right* to record the relative information ([1], pp143-147), with 8, 15 and 15 elements respectively (while $15 = 8 + (8 - 1)$). Let *column*[i] be 1 if there is a queen in the ith column of

*This work is partially supported by China 973 project (NKBRSF) and China Nature Science Foundation (No. 69873003).

the board, and 0 otherwise. If there is a queen in position (i, j), then $left[i+j]$ and $right[7-j+i]$ will be 1, and otherwise 0 (Suppose the array indexes start from 0).

With these arrays, safety-checking is direct and efficient. For test of position (i', j'), we need only to check $column[i']$, $left[i'+j']$ and $right[7-j'+i']$. If all the three array elements are 0, position (i', j') is safe for a new queen. A searching and back-tracking procedure can be used to find a safety placement, or all possible placements.

We can use a *bit-vector* to represent a series of states with two possible values for each state. Naturally, three bit-vectors are also enough for the *controlled-array method.* However, *array-indexing* is not an inherent operation on bit-vectors. We will to develop a pure bit-vector solution for n-queen problems, with only bit-vector operations. We'll stay in the view, that looking for a solution with general search-and-backtrack strategy is a series of tests for the queen-placement from the 1th row to the 8th row.

2 Bit-Vector Encoding

Suppose b_1 is the bit-vector for the first row, and the jth bit in b_1 has special value for the placed queen. The $(j-1)$th position in the second row is controlled by this queen, which can determined by shifting b_1 one-bit left. Similarly, $(j+1)$th position in the second row (also controlled by this queen) can be determined by right-shifting b_1. The controlled positions in the second row are represented by (We use $\ll$ and $\gg$ for left and right shifting, & for bit-wise *and*, and use 0 for the occupied/controlled positions, 1 for a free positions, respectively):

$$b_1 \ \& \ (b_1 \ll 1) \ \& \ (b_1 \gg 1)$$

It is also easy to find positions controlled by the first queen in the third row: left or right shifting b_1 one-bit more. That is:

$$b_1 \ \& \ (b_1 \ll 2) \ \& \ (b_1 \gg 2)$$

Generally, in the kth row, the positions controlled by the first queen can be determined by shifting b_1 left or right $k-1$ bits respectively, and bit-wise *and* these bit-vectors together. There are at most three positions in each row below controlled by the queen of the first row (Note, the bits can be shift out of the range).

$$B_1[k] \equiv b_1 \ \& \ (b_1 \ll (k-1)) \ \& \ (b_1 \gg (k-1))$$

We call b_1 the control-vector of the first queen.

Additional queens in following rows can be handled in the same way. Let's consider the second queen. Suppose the control-vector of this queen is b_2. The positions in the kth row $(k > 2)$ controlled by the second queen can be calculated:

$$B_2[k] \equiv b_2 \ \& \ (b_2 \ll (k-2)) \ \& \ (b_2 \gg (k-2))$$

Generally, the positions in the kth row controlled by the ith queen $(k > i)$ are:

$$B_k[k] \equiv b_i \ \& \ (b_i \ll (k-i)) \ \& \ (b_i \gg (k-i))$$

There are different ways to represent 8-queen problem with bit-vectors. One direct idea is using 8 occupation-vectors, each for a row. In a vector, there is only a bit in the different state from the others, to denote the position of the queen in the row.

This scheme has a weak point: at any step, the controlled columns by queens on board should be calculated from all the bit-vectors at hand. We adopt another way, using controlled-columns vectors. Each vector has one bit different from the vector just above, to denote the place of a queen in the row. This representation makes the column-checking easy.

Further, there is a natural way to join all these control-vectors into three working vectors. Let's name these bit-vectors *down*, *left* and *right* respectively.

Vector *down* represents the controlled-columns in the progress. When a new queen is placed on board, corresponding bit of *down* should be flipped to reflect the new situation.

Vector *left* represents the controlled positions by all queens above to the down-left diagonal direction. For every step down a row, *left* will be shifted one bit left. When a new queen is placed on the board, the bit of *left* should be flipped too. Vector *right* plays the same role as *left*, but for the right-down diagonal direction.

This representation goes very well with the search-and-backtrack procedure. The procedure starts from the first row, with all the three bit-vectors set to a pattern of none controlled position. In every row, the controlled positions are determined by:

$$B \equiv \mathit{down}\ \&\ \mathit{left}\ \&\ \mathit{right}$$

while 0 bits represents that the position is controlled by some queen on the board.

New queen is added to some position where the bit of B is 1, and all of the three working vectors are updated with the bit flipped. After that, the procedure goes into next step with bit-vectors *left* and *right* shifted. In this case, we can keep the invariant:

$$\mathit{controlled\text{-}positions} = \mathit{down}\ \&\ \mathit{left}\ \&\ \mathit{right}$$

in every row. This invariant guarantees that the procedure is correct.

It is easy to implement this search-and-backtrack process by a recursive procedure. The following is a general C function to illustrate the idea. Here the bit-vectors are represented by three `unsigned ints`, `d` for *down*, `l` for *left*, `r` for *right*. We don't need to include a test for recursive depth, because, when enough queens exist on the board, `d` will become a zero-vector eventually.

```
void nqueen (unsigned d,
             unsigned l, unsigned r)
{
   unsigned i = 1, t = d & l & r;
   if (d == 0)
   {/* solution, do something */
      return;
   }
   while (t != 0) {
      if (i & t) {
          nqueen(d ^ i, (l ^ i)<<1,
                 (r ^ i)>>1);
          t ^= i;
      }
      i <<= 1;
   }
}
```

Another modification to our original idea is the action `t ^= i`. It turns off the tested bit (it is 1 before). With this strategy, we can cut off some fruitless parts of the search tree. This function looks for all possible solutions. It is not hard to modify it for finding only one solution, or write a non-recursive procedure following this way.

The function is extremely short. It could be directly used for every n less than the number of bits in some integer types. If we really want to search in a big chess board, an extended bit-vector facility should be implemented first.

It is interesting to use this problem to test implementation of general bit-vector libraries, such as that in an implementation of C++ STL. Based on this bit-vector coding method, it is possible to design various parallel searching procedures for the n-queen problems.

References

[1] Wirth N. Algorithms + Data Structures = Programs, Prentice-Hall, Englewood Cliffs, NJ., 1976.